

Airtest Remote Control System – Architecture & Design Document

1. 系統總覽 (System Overview)

本系統旨在實現跨網域遠端 Android 自動化測試。主要元件包含：

1. Cloud Server (FastAPI + WebSocket + Task Queue)
2. Pi Agent (Raspberry Pi 3B)
3. Web UI (React + WebRTC)
4. WebRTC Media Pipeline (scrcpy H.264 → Browser)
5. Airtest 任務觸發系統

系統允許使用者直接於瀏覽器即時控制 Android 裝置，無需 VPN、無需安裝 Tailscale、無需 Root。

2. 系統架構 (System Architecture)

整體架構採星狀拓撲，由 Cloud Server 擔任中樞：

Web UI $\rightleftharpoons$ Cloud Server $\rightleftharpoons$ Pi Agent $\rightleftharpoons$ Android Device

Cloud Server 提供：

- REST API
- WebSocket signaling
- Airtest 任務管理
- Agent 狀態監控
- 前端 UI 靜態檔案服務

Pi Agent 提供：

- ADB 控制 (USB / WiFi Fallback)
- scrcpy H.264 encoder
- WebRTC RTP 推流
- Airtest 執行環境

Web UI 提供：

- 即時視訊操控 (WebRTC)
- 裝置狀態檢視
- 執行 Airtest 腳本
- 下載 Log / Report

3. Cloud Server 架構

Cloud Server 使用 FastAPI + WebSocket，功能包含：

- Agent 註冊與心跳 (Register / Heartbeat)
- signaling server (WebRTC)
- REST API for UI

- 任務佇列管理（每台 Pi 一條 Queue）
- Airtest script 檔案儲存與下載
- Web UI 靜態檔案托管

運作流程：

1. Pi Agent 啟動後連線至 ws://server:8081/ws/agent/{id}
2. UI 在瀏覽器開啟 ws://server:8081/ws/ui/{id}
3. Cloud Server 進行 Offer / Answer / ICE 交換
4. 雙方建立 WebRTC 連線完成串流

4. Pi Agent 架構

Pi Agent 核心模組：

- adb_manager.py
 - 自動偵測 USB Android
 - 若無 USB → WiFi adb connect
 - 提供 input tap / swipe / key API
- webrtc_streamer.py
 - srcpy --video-codec=h264
 - RTP 封包送進 WebRTC PeerConnection
 - DataChannel → ADB input
- websocket_client.py
 - 與 Cloud Server 保持持續連線
 - 收發 signaling 與 Task 指令
- airtest_runner.py
 - 執行單支 Airtest
 - 收集 Screenshot、Log、Report
- device_detector.py
 - 監測 ADB 裝置連線狀態

5. Web UI 架構

Web UI 使用 React，核心功能包含：

- Agents Dashboard：顯示所有 Pi 狀態
- Device Control：WebRTC player embedded
- DataChannel input：滑鼠與鍵盤事件轉成 commands
- Airtest Management：上傳、執行、查看結果
- Logs Viewer：即時顯示 Pi 回傳訊息

所有操作皆透過 Cloud Server，不需直連 Pi。

6. WebRTC 串流原理

本系統採用：

- srcpcy → H.264 → ffmpeg pipeline（視需要）
- WebRTC RTCPeerConnection 傳送 H.264 RTP

WebRTC 提供：

- NAT 穿透（ICE + STUN）
- 低延遲影像傳輸（~150 ms）
- 雙向 DataChannel（控制命令）

Cloud Server 僅處理 signaling，不參與媒體傳輸，因此延遲極低。

7. Airtest 任務流程

完整的 Airtest workflow：

1. 使用者於 Web UI 上傳 .air script
2. Cloud Server 儲存腳本並加入指定 Pi 的 Queue
3. Pi Agent 收到任務後執行：
airtest run script.air --device adb://127.0.0.1:5555
4. 過程中產生 screenshot / log
5. 完成後上傳回 Cloud
6. UI 顯示結果

每台 Pi 一次只能跑一支 Airtest（任務佇列保證順序）。

8. 跨網域原理 (No VPN Required)

因 WebRTC 具有 NAT Traversal 能力，因此不需要 Tailscale 或 VPN。

流程：

1. UI 與 Pi 皆與 Cloud Server 建立 WebSocket
2. WebRTC Offer / Answer / ICE 透過 WebSocket 傳遞
3. 若 P2P 成功 → 直連
4. 若失敗 → 使用 TURN（可選）

因此：

- 不需同網段
- 不需固定 IP
- 不需安裝任何 Android App
- 適用於 4G / 5G / WiFi / 多企業網路

9. 模組化設計

整套系統拆分為三大 Repository：

1. Cloud Server
2. Pi Agent

3. Web UI

可獨立開發、測試與部署，使架構高度彈性且可維護。

10. 結語

此架構已成功整合：WebRTC + Airtest + scrcpy + Cloud Queue + Web UI，並能達成跨網域、低延遲、高穩定度的 Android 遠端控制與自動化測試。

可用於：

- Robot QA
- Android Function Test
- 遠端維運（保養 / Debug）
- 排程測試環境

本文件可作為 VS Code GPT Agent 生成程式碼之基礎架構參考。